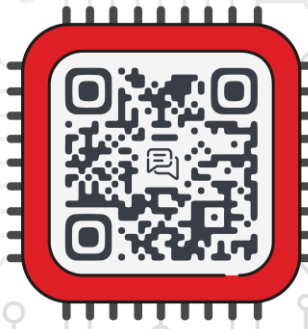




3



14



1680 sp

عملي

كلية الهندسة المعلوماتية

السنة الثانية

Abstract & Interface Review

عملي مشترك



RB Informatics; 10/11/2024

برمجة ٣

■ ملاحظة اربيسيزية (ما هو التجريد):

لن نعطي تعريفاً تقليدياً لكن ببساطة المستخدم لا يهتم بـ "كيف" يقوم البرنامج خاصتك بالعمل، بل هو يهتم بـ "ما" يمكن لبرنامجك فعله. مثلاً عند استخدام sorting method ضمن برنامجك، يمكن أن يتم عملها بألف طريقة لكن المستخدم لن يهتم سوى بأن الـ method تقوم بعمل sort لقائمة من البيانات ولا تهمه الطريقة (ربما تفعلها بـ bubble sort, merge sort or quick sort الخ). فكمطورين يجب علينا أن نعرض للمستخدم ما يمكنه فعله من خلال التطبيق الذي نطوره دون الحاجة لإشغاله بالتفاصيل، وهذا هو لب التجريد.

لتحقيق مفهوم التجريد أثناء استخدام الـ OOP لدينا طريقتان:
abstract classes and interfaces

■ الـ abstract class يتسم بالخصائص الآتية:

1. لا يمكن عمل object منه مباشرة.
2. يمكن أن يوجد به abstract methods و concrete methods (دوال عادية غير مجردة).
(abstract methods are methods without a body{ })
3. يمكنه أن يحوي متغيرات static and non-static.
4. كل class يرث من الـ abstract class يتوجب عليه عمل override للـ abstract method التي ضمن الـ abstract class.
5. يمكن أن يحوي بواني.

■ الـ interface و هو blueprint تشبه الـ class لكن تتسم بالتالي:

1. لا يمكن عمل object منها مباشرة .
2. كل method داخلها هي abstract بالضرورة by default.
3. كل الخصائص التي يتم تعريفها ضمنها هي by default (final , static).
4. الـ classes التي تقوم بعمل implement للـ interface يتوجب عليها أن تعمل override لكافة الـ methods الموجودة ضمن الـ interface التي ليست default.
5. يمكن للـ class الواحد عمل implement لأكثر من interface.
6. يمكن للـ interface أن ترث من أكثر من interface ((implicit multiple inheritance)).
7. لا يمكن أن يحوي أي باني.

■ إثراء :

في نسخ جافا 8 java فما بعد أصبح يمكنك وضع methods لها body ضمن الـ interface باستخدام keyword و هي default ، و يمكنك كذلك وضع static methods ضمنها ، و هذا يؤدي لتأمين ميزة تجعل هنالك توازناً بين المرونة ضمن الكود و التجريد.

حيث تم اضافتها لعدة أسباب، منها:

في بعض الحالات يكون لديك كثير من الـ classes التي تقوم بعمل implement للـ interface ، فلو فرضاً وصلت لمرحلة تريد إضافة تابع جديد للـ interface و هذا التابع ينفذ سلوكاً متشابهاً بين العديد من الـ classes الكثيرة التي تقوم بتحقيق الـ interface، عندها يمكنك استخدام default لجعل التابع طبيعياً و ثم عمل override لهذا التابع عند الحاجة في الـ classes التي يختلف سلوك هذا التابع فيها ، و هذا يجعل عمل أي تعديل على الكود ككل غير ضروري ، و يتم إضافة التابع فحسب إلى الـ interface.

كذلك الـ static methods تقوم بتأمين بعض الوظائف التي تتطلب الوصول لها مباشرة دون إنشاء كائنات، و هذا أمن مرونة أكبر أثناء استخدام الـ interface، بعبارة أخرى تم وضع هذه الميزة لتمكين المطورين من التحكم الأكبر بالـ interface و إضافة توابع لها دون التعديل الكبير على كل class يرث منها ، و هذا لا يغير من غرض الـ interface و ميزاتها التي تساعد على التجريد لكن أضاف لها ميزات إضافية.

كيف لما سبق أن يحقق التجريد ؟

التجريد بالنسبة الأكبر يتحقق من خلال الـ `interface` لأن كل توابعها `abstract` بالمجمل، مما يسمح لها بتحديد مجموعة من الوظائف التي سيقوم بها أي `class` يقوم بعمل `implement` لها .
حيث هي بهذا الأمر تحدد " ما هي " الوظائف التي يجب على هذا `class` القيام بها دون تحديد " كيفية " قيامه بها .

بينما الـ `abstract class` يمكن له القيام بفعل ذلك لكن ليس بالضرورة بنسبة كبيرة، أي أنه يحدد " ما هي " (بعض) الوظائف التي يجب على `class` الذي يقوم بعمل `extend` له القيام بها ، لكن قد يتم تعريف بعض القواسم المشتركة بين عدة `classes` ضمن الـ `abstract class` و تحديد كيفية عملها و تعريف خصائص لا علاقة لها بالمفهوم، وهذا يعني أن نسبة التجريد هنا أقل من الـ `interface`.

السؤال الأول:

عرف 4 CLASSES

- Abstract Class Person:

attributes: name -gender -birthday- ismarried

methods: parameterized constructor – set&get for each attribute and abstract method `displayInfo()`

- Class Student

extends Person

attributes: specialization

methods: parameterized constructor and implement the `displayinfo()` abstract method

- Class Employee

extends Person

attributes:workPlace

methods: parameterized constructor and implement the `displayinfo()` abstract method

- Then in the **Main Class** make the main method and create an object of employee and student, then use the `displayInfo()` for each of them.

```
1 // Person.java file
2 // Person Class
3 // we put abstract here to make it
  abstract class
4 public abstract class Person {
5     //attributes
6     String name;
7     String gender;
8     String birthday;
9     boolean isMarried;
10    //constructor
11    public Person(String n, String g, String
  b, boolean i) {
12        name = n;
13        gender = g;
14        birthday = b;
15        isMarried = i;
16    }
17    //getters and setters
18    public String getName() {
19        return name;
20    }
21    public String getGender() {
22        return gender;
23    }
24    public String getBirthday() {
25        return birthday;
26    }
27    public boolean getIsMarried() {
28        return isMarried;
29    }
30    public void setName(String n) {
31        name = n;
32    }
33    public void setGender(String g) {
34        gender = g;
35    }
36    public void setBirthday(String b) {
37        birthday = b;
38    }
39    public void setIsMarried(boolean i) {
40        isMarried = i;
41    }
42    // the abstract method
43    public abstract void displayInfo(); }
```

```
1 //Student.java file
2 // Student Class
3 public class Student extends Person {
4     //attribute
5     String specialization;
6     //constructor
7     public Student(String n, String g, String b, boolean i,
  String s) {
8         super(n, g, b, i);
9         specialization = s;
10    }
11    //overriding the abstract method
12    @Override
13    public void displayInfo() {
14        System.out.println("Name: " + getName());
15        System.out.println("Gender: " + getGender());
16        System.out.println("Birthday: " + getBirthday());
17        System.out.println("Specialization: " +
  specialization);
18        System.out.println("-----");
19    }}
```

```
1 //Employee.java file
2 // Employee Class
3 public class Employee extends Person {
4     //attribute
5     String workPlace;
6     //constructor
7     public Employee(String n, String g, String b, boolean i,
  String w) {
8         super(n, g, b, i);
9         workPlace = w;
10    }
11    //overriding the abstract method
12    @Override
13    public void displayInfo() {
14        System.out.println("Name: " + getName());
15        System.out.println("Gender: " + getGender());
16        System.out.println("Birthday: " + getBirthday());
17        if (getIsMarried() == true) System.out.println("is
  Married: yes");
18        else System.out.println("is Married: no");
19        System.out.println("Work place: " + workPlace);
20        System.out.println("-----");
21    }}
```

```
1 // Main.java file
2 //Main Class
3 public class Main {
4     public static void main(String[] args) {
5         //instance of student
6         Student s = new Student("Fadi",
7             "Male", "1999", false, "Computer Science");
8         s.displayInfo();
9         //instance of employee
10        Employee e = new Employee("Rana",
11            "Female", "1986", true, "Damas University");
12        e.displayInfo();
13    }}
```



السؤال الثاني :

4 classes :

- Abstract class Shape

has 3 abstract methods:

حساب المحيط getPerimeter()

حساب المساحة getArea()

طباعة المعلومات printInfo()

- Circle class extends Shape

has radius attribute and parameterized constructor
and override the methods of shape

- Rectangle class extends Shape

has height and width attributes and parameterized constructor.
and override the methods of shape

- Square class extends Shape

has side attribute and parameterized constructor
and override the methods of shape

- Main class

فيه التابع main وضمنه سننشئ reference من Shape class ، ثم ستيح للمستخدم اختيار الشكل الذي يود إدخاله، ثم إدخال معلوماته واستدعاء التابع printInfo لطباعته.

```
1 // Shape Class
2 abstract public class Shape {
3     abstract public void printInfo() ;
4     abstract public double getArea();
5     abstract public double getPerimeter();
6 }
```

```
1 //Circle Class
2 public class Circle extends Shape {
3     //attribute
4     private double radius;
5     //parameterized constructor
6     public Circle(double radiusValue) {
7         radius = radiusValue;
8     }
9
10    public double getRadius() {
11        return radius;
12    }
13    public double getDiameter() {
14        return 2 * radius;
15    }
16    @Override
17    public double getPerimeter() {
18        return Math.PI * getDiameter();
19    }
20    @Override
21    public double getArea() {
22        return Math.PI * radius * radius;
23    }
24    @Override
25    public void printInfo() {
26        System.out.println(" Radius = " + radius + "
27        , Area = " + getArea() + " , Circumference= " +
28        getPerimeter());
29    }
30 }
```

```
//Rectangle Class
public class Rectangle extends Shape {
    // attributes
    private double width;
    private double height;
    //parameterized constructor
    public Rectangle(double width, double height)
    {
        this.width = width;
        this.height = height;
    }

    public double getHeight() {
        return height;
    }

    public double getWidth() {
        return width;
    }
    @Override
    public double getPerimeter() {
        return 2 * (width + height);
    }
    @Override
    public double getArea() {
        return height * width;
    }
    @Override
    public void printInfo() {
        System.out.println(" Height = " +
        getHeight() + " Width = " + getWidth() + " , Area
        = " + getArea() + " , Perimeter= " +
        getPerimeter());
    }
}
```



```

2 // including the Scanner within this java file
3 import java.util.Scanner;
4 //Main Class
5 public class Main {
6 public static void main(String[] args) {
7     //scanner object
8     Scanner input = new Scanner(System.in);
9     //shape reference
10    Shape shape;
11    double x, y;
12    System.out.println(" Enter:\n 1 for Circle \n 2 for Rectangle
\n 3 for Square");
13    // نختار الرقم بناء على ما نود إنشائه
14    int choice = input.nextInt();
15    switch (choice) {
16        case 1:
17            System.out.print(" Enter Circle Information : Radius = ");
18            // إدخال نصف القطر
19            x = input.nextDouble();
20            //إسناد كائن من نوع دائرة للـ shape reference
21            shape = new Circle(x);
22            // طباعة معلومات الدائرة
23            shape.printInfo();
24            break;
25        case 2:
26            System.out.print(" Enter Rectangle Information : width
= ");
27            // العرض
28            x = input.nextDouble();
29            System.out.print(" height = ");
30            // الطول
31            y = input.nextDouble();
32            // إسناد كائن من نوع مستطيل للـ shape reference
33            shape = new Rectangle(x, y);
34            // طباعة معلومات المستطيل
35            shape.printInfo();
36            break;
37        case 3:
38            System.out.print(" Enter square Information : side = ");
39            // طول الضلع
40            x = input.nextDouble();
41            //إسناد كائن من نوع مربع للـ shape reference
42            shape = new Square(x);
43            shape.printInfo();
44            break;
45        default:
46            //في حالة إدخال رقم غير 1 او 2 او 3
47            System.out.println("You entered wrong number ");
48    }
}

```

```

1 public class Square extends
Shape{
2     //attribute
3     private double side;
4     //parameterized constructor
5     public Square(double side) {
6         this.side = side;
7     }
8     public double getSide() {
9         return side;
10    }
11    @Override
12    public double getPerimeter() {
13        return side*4;
14    }
15    @Override
16    public double getArea() {
17        return side*side;
18    }
19    @Override
20    public void printInfo() {
21        System.out.println(" side = "
+ getSide() + " , Area = " + getArea()
+ " , Perimeter= " + getPerimeter());
22    }
}

```



السؤال الثالث :

عرف 3 classes و interface

- interface Bank
- has one method called rateOfInterest

لعرض نسبة الفائدة

- class BSF implements Bank
- overrides this method to return 5.35
- class Audi implements Bank
- overrides this method to return 5.95
- class Main
- has the main method:

نقوم بجعل المستخدم يدخل إما 1 لعرض نسبة الفائدة للبنك BSF
أو يدخل 2 لعرض نسبة الفائدة للبنك Audi

```
1 //Bank Interface
2 public interface Bank {
3     float rateOfInterest();
4 }
```

```
1 //BSF class
2 public class BSF implements Bank{
3     @Override
4     public float rateOfInterest() {
5         return 5.35f;
6     }
7 }
```

```
1 //Audi Class
2 public class Audi implements Bank{
3     @Override
4     public float rateOfInterest() {
5         return 5.95f;
6     }
7 }
```

```
1 import java.util.Scanner;
2 //Main Class
3 public class Main {
4     public static void main(String[] args) {
5         Scanner input = new Scanner(System.in);
6         // reference للبنك
7         Bank bank;
8         System.out.println (" Enter:\n 1 for rate of interest in
9         BSF Bank \n 2 for rate of interest in Audi Bank ");
10        // ندخل الرقم بناء على البنك المختار
11        int choice=input.nextInt();
12        switch (choice){
13            case 1:
14                // ننشئ اوبجكت من البنك و نسنده للreference
15                bank=new BSF();
16                System.out.println("BSF:
17                "+bank.rateOfInterest());
18                break;
19            case 2:
20                // ننشئ اوبجكت من البنك و نسنده للreference
21                bank=new Audi();
22                System.out.println("Audi:
23                "+bank.rateOfInterest());
24                break;
25            default:
26                System.out.println ("You entered wrong number");
27        }
28    }
29 }
```

السؤال غير المحلول:
سنقوم بحله بطريقتين:
الطريقة الأولى:

4 Classes

abstract class Building

has three attributes :BuildingNumber -floorArea-numberOfFloors
and abstract method printInfo

class School extends Building

with attributes: classesPerFloor, schoolName
override printInfo method

class Hospital extends Building

with attributes: numberOfDepartments, numberOfBeds, hospitalName
override printInfo method

Main Class

with the main method, and make arraylist of type Building, make the user enter schools and hospitals, add them to the arrayList then print their info.

```
1 //abstract Building Class
2 public abstract class Building {
3     //attributes
4     double floorArea;
5     int buildingNumber;
6     int numberOfFloors;
7     // parameterized
8     Building (double
9     floorArea,int buildingNumber,int
10    numberOfFloors){
11        this.floorArea = floorArea;
12        this.buildingNumber
13        =buildingNumber;
14        this.numberOfFloors =
15        numberOfFloors;
16    }
17    // abstract method to print
18    info
19    abstract void printInfo();
20 }
```

```
1 //School Class
2 public class School extends Building{
3     int classesPerFloor;
4     String schoolName;
5
6     School (double floorArea,int buildingNumber,int
7     numberOfFloors,int classesPerFloor,String schoolName){
8         super(floorArea,buildingNumber,numberOfFloors);
9         this.classesPerFloor = classesPerFloor;
10        this.schoolName =schoolName;
11    }
12    @Override
13    public void printInfo() {
14        System.out.println("school info");
15        System.out.println("school name is " + schoolName);
16        System.out.println("the building number is " +
17        buildingNumber);
18        System.out.println("floor area " + floorArea);
19        System.out.println("number of floors " + numberOfFloors);
20        System.out.println("number of classes per floor is
21        "+classesPerFloor);
22        System.out.println("-----"); } }
```

```
1 //Hospital Class
2 public class Hospital extends Building{
3     int numberOfBeds;
4     int numberOfDepartments;
5     String hospitalName;
6
7     Hospital (double floorArea,int buildingNumber,int numberOfFloors,int
numberOfBeds,int numberOfDepartments,String hospitalName){
8         super(floorArea,buildingNumber,numberOfFloors);
9         this.numberOfBeds = numberOfBeds;
10        this.numberOfDepartments= numberOfDepartments;
11        this.hospitalName = hospitalName;
12    }
13    @Override
14    public void printInfo() {
15        System.out.println("hospital info");
16        System.out.println("hospital name is "+hospitalName);
17        System.out.println("the building number is " + buildingNumber);
18        System.out.println("floor area " + floorArea);
19        System.out.println("number of floors " + numberOfFloors);
20        System.out.println("departments number is " +numberOfDepartments);
21        System.out.println("beds number is " + numberOfBeds);
22        System.out.println("-----");
23    }
24 }
```

```
1 import java.util.ArrayList;
2 import java.util.Scanner;
3 //Main Class
4 public class Main {
5     public static void main(String[] args) {
6         //arraylist of buildings
7         ArrayList<Building> buildings = new ArrayList<>();
8         Scanner scanner = new Scanner(System.in);
9         System.out.println("enter the number of buildings do you want to enter");
10        //number of buildings
11        int num = scanner.nextInt();
12        for(int i= 0; i<num;i++)
13        {
14            //entering the details according to the building type
15            // then makes an object and add it to the list
16            System.out.println("enter the building number");
17            int buildingNumber = scanner.nextInt();
18            System.out.println("enter the number of floors");
19            int numberOfFloors = scanner.nextInt();
```

```

20    System.out.println("floor area");
21    double floorArea = scanner.nextDouble();
22    System.out.println("the type of building \n 1-school \n 2-hospital");
23    int t=scanner.nextInt();
24    switch (t)
25    {
26        case 1 :
27            scanner.nextLine();
28            System.out.println("the school name");
29            String schoolName =scanner.nextLine();
30            System.out.println("classes per floor");
31            int classesPerFloor =scanner.nextInt();
32            // عملنا object ضمن أقواس add بالبيانات المعطاة مباشرة
33            buildings.add(new
School(floorArea,buildingNumber,numberOffFloors,classesPerFloor,schoolName));
34            break;
35        case 2 :
36            scanner.nextLine();
37            System.out.println("enter the hospital name");
38            String hospitalName= scanner.nextLine();
39            System.out.println("enter the departments number");
40            int numberOfDepartments = scanner.nextInt();
41            System.out.println("enter beds number");
42            int numberOfBeds = scanner.nextInt();
43            // عملنا object ضمن أقواس add بالبيانات المعطاة مباشرة
44            buildings.add(new Hospital
(floorArea,buildingNumber,numberOffFloors,numberOfBeds,numberOfDepartments,hospital
Name));
45            break;
46        default:
47            System.out.println("wrong input");
48    }
49 }
50 // foreach للمرور على كل المباني و استدعاء تابع طباعة البيانات
51 for (Building building : buildings)
52 {
53     building.printInfo();
54 }

```



الطريقة الثانية المطلوبة :

Make Building interface instead of abstract class

ضع الـ attributes الخاصة بالمبنى بكل الكلاسين الذين يحققان الـ interface السابقة، أي ضع - floorArea
 BuildingNumber - numberOfFloors ضمن كلا الكلاسين كذلك و غير ما يلزم ثم فقط قم بعمل
 override printInfo ، وباقي الخطوات ضمن التابع الرئيسي تبقى على حالها.

```
1 //Building Interface
2 public interface Building {
3     void printInfo();
4 }
```

```
1 //School Class
2 public class School implements Building{
3     double floorArea;
4     int buildingNumber;
5     int numberOfFloors;
6     int classesPerFloor;
7     String schoolName;
8     School (double floorArea,int
buildingNumber,int numberOfFloors,int
classesPerFloor,String schoolName){
9         this.floorArea = floorArea;
10        this.buildingNumber =buildingNumber;
11        this.numberOfFloors = numberOfFloors;
12        this.classesPerFloor = classesPerFloor;
13        this.schoolName =schoolName;
14    }
15    @Override
16    public void printInfo() {
17        System.out.println("school info");
18        System.out.println("school name is " +
schoolName);
19        System.out.println("the building number
is " + buildingNumber);
20        System.out.println("floor area " +
floorArea);
21        System.out.println("number of floors " +
numberOfFloors);
22        System.out.println("number of classes
per floor is "+classesPerFloor);
23        System.out.println("-----");
24    }}
```

```
1 //Hospital Class
2 public class Hospital implements Building{
3     double floorArea;
4     int buildingNumber;
5     int numberOfFloors;
6     int numberOfBeds;
7     int numberOfDepartments;
8     String hospitalName;
9     Hospital (double floorArea,int
buildingNumber,int numberOfFloors,int
numberOfBeds,int numberOfDepartments,String
hospitalName){
10        this.floorArea = floorArea;
11        this.buildingNumber =buildingNumber;
12        this.numberOfFloors = numberOfFloors;
13        this.numberOfBeds = numberOfBeds;
14        this.numberOfDepartments=
numberOfDepartments;
15        this.hospitalName = hospitalName;
16    }
17    @Override
18    public void printInfo() {
19        System.out.println("hospital info");
20        System.out.println("hospital name is
"+hospitalName);
21        System.out.println("the building number
is " + buildingNumber);
22        System.out.println("floor area " +
floorArea);
23        System.out.println("number of floors " +
numberOfFloors);
24        System.out.println("departments number
is "+numberOfDepartments);
25        System.out.println("beds number is " +
numberOfBeds);
26        System.out.println("-----");
27    }}
```

```
1 import java.util.ArrayList;
2 import java.util.Scanner;
3 //Main Class
4 public class Main {
5     public static void main(String[] args) {
6         //arraylist of buildings
7         ArrayList<Building> buildings = new ArrayList<>();
8         Scanner scanner = new Scanner(System.in);
9         System.out.println("enter the number of buildings do you want to enter");
10        //number of buildings
11        int num = scanner.nextInt();
12        for(int i= 0; i<num;i++)
13        {
14            //entering the details according to the building type
15            // then makes an object and add it to the list
16            System.out.println("enter the building number");
17            int buildingNumber = scanner.nextInt();
18            System.out.println("enter the number of floors");
19            int numberOfFloors = scanner.nextInt();
20            System.out.println("floor area");
21            double floorArea = scanner.nextDouble();
22            System.out.println("the type of building \n 1-school \n 2-hospital");
23            int t=scanner.nextInt();
24            switch (t)
25            {
26                case 1 :
27                    scanner.nextLine();
28                    System.out.println("the school name");
29                    String schoolName =scanner.nextLine();
30                    System.out.println("classes per floor");
31                    int classesPerFloor =scanner.nextInt();
32                    // عملنا object ضمن أقواس add بالبيانات المعطاة مباشرة
33                    buildings.add(new
School(floorArea,buildingNumber,numberOfFloors,classesPerFloor,schoolName));
34                    break;
35                case 2 :
36                    scanner.nextLine();
37                    System.out.println("enter the hospital name");
38                    String hospitalName= scanner.nextLine();
39                    System.out.println("enter the departments number");
40                    int numberOfDepartments = scanner.nextInt();
41                    System.out.println("enter beds number");
42                    int numberOfBeds = scanner.nextInt();
```

```
43          // عملنا object ضمن أقواس add بالبيانات المعطاة مباشرة
44          buildings.add(new
Hospital(floorArea,buildingNumber,numberOfFloors,numberOfBeds,numberOfDepartments,hospitalN
ame));
45          break;
46          default:
47              System.out.println("wrong input");
48          }
49      }
50      // foreach للمرور على كل المباني و استدعاء تابع طباعة البيانات
51      for (Building building : buildings)
52      {
53          building.printInfo();
54      }}
```

See you

